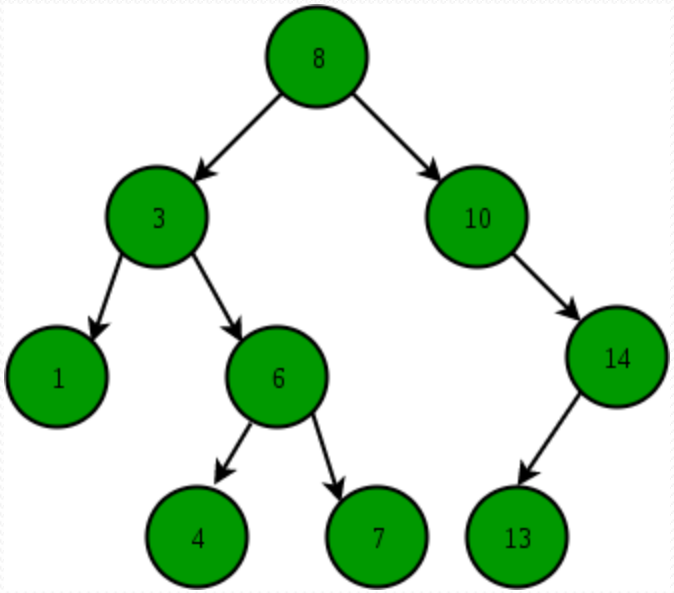
Binary Search Tree

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.



Insertion of a key

```
      100                                  100
     /   \        Insert 40              /    \
   20     500    --------->            20     500
  /  \                                 /  \
 10   30                              10   30
                                             \
                                              40
```

50, 30, 20, 40, 70, 60,80

50, 30, 20, 40, 70, 60,80

50

30 70

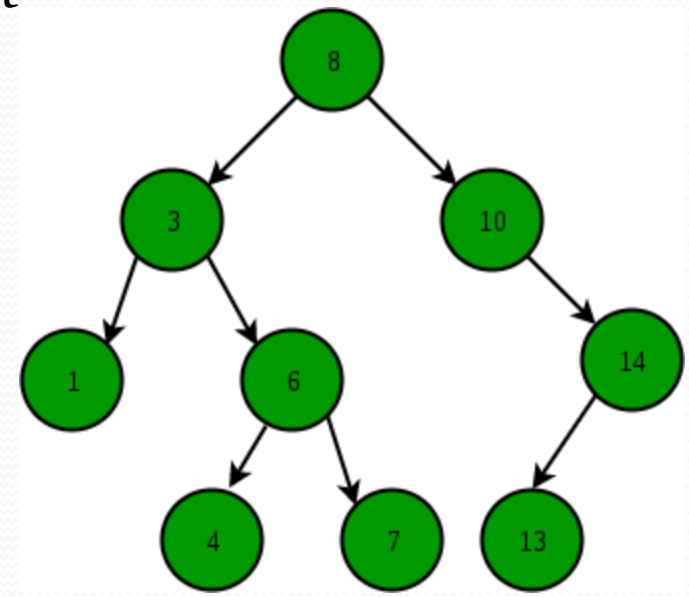
20 40 60 80

```
struct node
{
    int key;
    struct node *left, *right;
};

struct node *newNode(int item)
{
    struct node *temp =  (struct node *)malloc(sizeof(struct node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}
```

```
struct node* insert(struct node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->key)
        node->left  = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}
```

Search

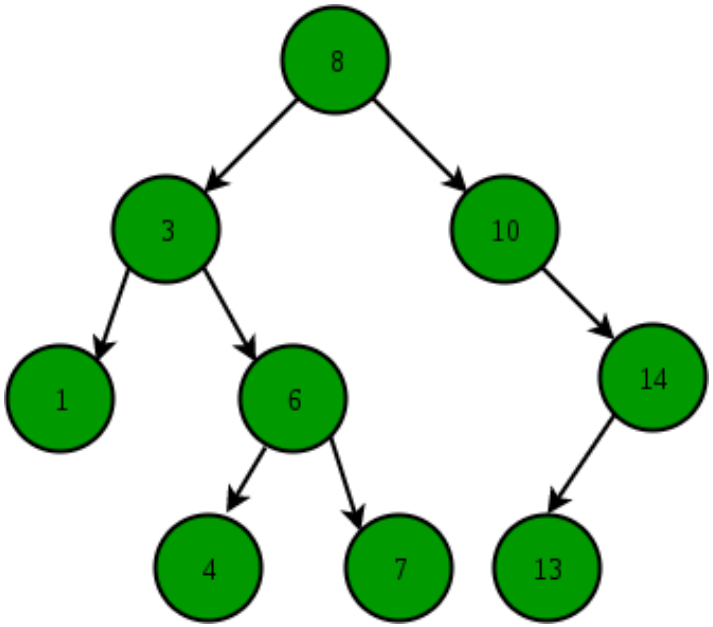
```
struct node* search(struct node* root, int key)
{
    // Base Cases: root is null or key is present at
root
    if (root == NULL || root->key == key)
       return root;

    // Key is greater than root's key
    if (root->key < key)
       return search(root->right, key);

    // Key is smaller than root's key
    return search(root->left, key);
}
```



- **Time Complexity:** The worst case time complexity of search and insert operations is O(h) where h is height of Binary Search Tree. In worst case, we may have to travel from root to the deepest leaf node. The height of a skewed tree may become n and the time complexity of search and insert operation may become O(n).

Traversal

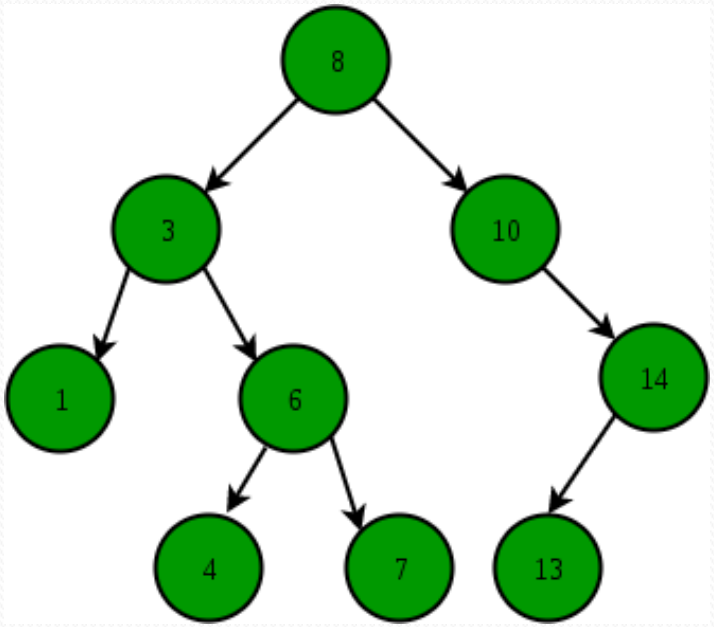
```
void inorder(struct node *root)
{
    if (root != NULL)
         inorder(root->left);
        printf("%d \n", root->key);
        inorder(root->right);
    } }
```



1 3 4 6 7 8 10 13 14

Traversal

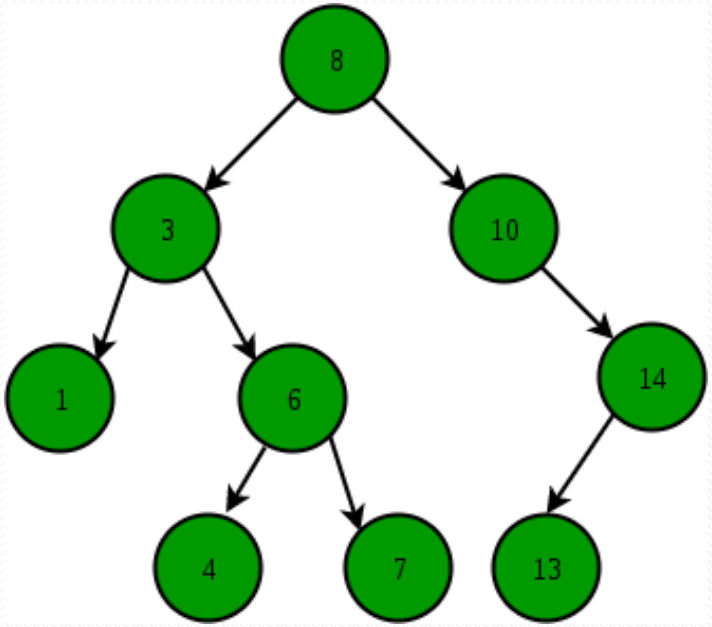
```
void preorder(struct node *root)
{
    if (root != NULL)
        printf("%d \n", root->key);
        preorder(root->left);
        preorder(root->right);
    }

}
```



8 3 1 6 4 7 10 14 13

Traversal

```
void postorder(struct node *root)
{
    if (root != NULL)
        postorder(root->left);
        postorder(root->right);
        printf("%d \n", root->key);
    }

}
```



1 4 7 6 3 13 14 10 8

Delete

1. Node to be deleted at leaf
2. *Node to be deleted has only one child*
3. *Node to be deleted has two children*

- Node to be deleted at leaf

```
        50                              50
     /     \         delete(20)       /   \
    30      70       --------->     30     70
   /  \    /  \                       \    /  \
 20   40  60   80                      40  60   80
```

Node to be deleted has only one child

```
      50                              50
    /     \         delete(30)      /   \
   30      70       --------->    40     70
     \    /  \                          /  \
     40  60   80                       60   80
```

- *Node to be deleted has two children*

```
      50                            60
   /     \         delete(50)      /   \
  40      70       --------->    40    70
         /  \                            \
        60   80                           80
```

```
struct node* deleteNode(struct node* root, int key)
{
    // base case
    if (root == NULL) return root;

    // If the key to be deleted is smaller than the root's key,
    // then it lies in left subtree
    if (key < root->key)
        root->left = deleteNode(root->left, key);

    // If the key to be deleted is greater than the root's key,
    // then it lies in right subtree
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
```

```
else
    {
        // node with only one child or no child
        if (root->left == NULL)
        {
            struct node *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL)
        {
            struct node *temp = root->left;
            free(root);
            return temp;
        }

        // node with two children: Get the inorder successor (smallest
        // in the right subtree)
        struct node* temp = minValueNode(root->right);

        // Copy the inorder successor's content to this node
        root->key = temp->key;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->key);
    }
    return root;
}
```

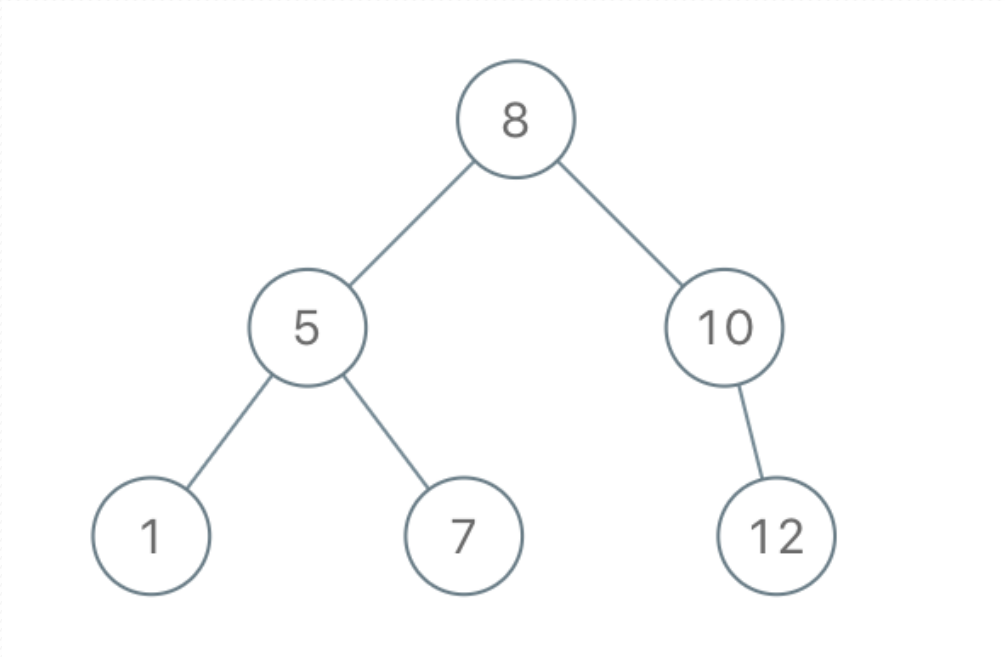
```
struct node * minValueNode(struct node* node)
{
    struct node* current = node;

    /* loop down to find the leftmost leaf */
    while (current && current->left != NULL)
        current = current->left;

    return current;
}
```

Construct tree

- Preorder-{10, 5, 1, 7, 40, 50}

```
      10
    /   \
   5     40
  /  \      \
 1    7      50
```

- [8,5,1,7,10,12]



- 50, 70, 60, 20, 90, 10, 40, 100

- The first element of preorder traversal is always root. We first construct the root.
- Then we find the index of first element which is greater than root. Let the index be ‘i’. The values between root and ‘i’ will be part of left subtree, and the values between ‘i+1’ and ‘n-1’ will be part of right subtree.
- Divide given pre[] at index “i” and recur for left and right sub-trees.
- **($O(n^2)$ time complexity)**

From postorder traversal

- {1, 7, 5, 50, 40, 10}

```
      10
    /    \
   5      40
  /  \      \
 1    7      50
```

{2, 6, 4, 9, 13, 11, 7},

```
      7
    /    \
   4      11
  / \    /  \
 2   6  9    13
```

Algorithm:

- Push root of the BST to the stack i.e, last element of the array.
- Start traversing the array in reverse, if next element is > the element at the top of the stack then,
  set this element as the right child of the element at the top of the stack and also push it to the stack.
- Else if, next element is < the element at the top of the stack then, start popping all the elements from the stack until either the stack is empty or the current element becomes > the element at the top of the stack.
- Make this element left child of the last popped node and repeat the above steps until the array is traversed completely.

from its given level order traversal

- {7, 4, 12, 3, 6, 8, 1, 5, 10}

```
        7
      / \
     4   12
    / \  /
   3  6 8
  /  /   \
 1  5    10
```

Count nodes

Iterative

1) Create an empty Queue Node and push root node to Queue.
2) Do following while nodeQueue is not empty.
   a) Pop an item from Queue and process it.

      a. 1) If it is full node then increment count++.

      b) Push left child of popped item to Queue, if available.

      c) Push right child of popped item to Queue, if available.

```
// If tree is empty
    if (!node)
        return 0;
    queue<Node *> q;

    // Do level order traversal starting from root
    int count = 0; // Initialize count of full nodes
    q.push(node);
    while (!q.empty())
    {
        struct Node *temp = q.front();
        q.pop();

        if (temp->left && temp->right)
            count++;

        if (temp->left != NULL)
            q.push(temp->left);
        if (temp->right != NULL)
            q.push(temp->right);
    }
    return count;
```

Time Complexity: O(n)

- **Recursive**

```
unsigned int getCount(struct Node* root)
{
    if (root == NULL)
        return 0;

    int res = 0;
    if  (root->left && root->right)
        res++;

    res += (getCount(root->left) +  getCount(root->right));
    return res;
}
```

Height

maxDepth()

1. If tree is empty then return 0
2. Else

(a) Get the max depth of left subtree recursively i.e., call maxDepth(tree->left-subtree)

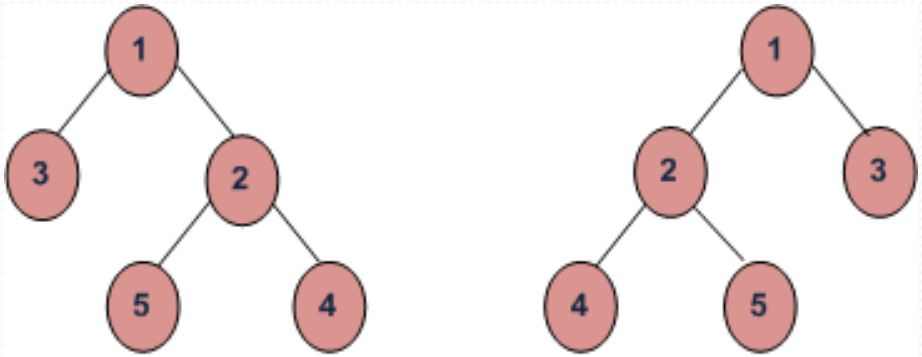
(b) Get the max depth of right subtree recursively i.e., call maxDepth(tree->right-subtree)

(c) Get the max of max depths of left and right subtrees and add 1 to it for the current node. max_depth = max(max dept of left subtree, max depth of right subtree) + 1

(d) Return max_depth

```
int maxDepth(node* node)
{
    if (node == NULL)
        return 0;
    else
    {
        /* compute the depth of each subtree */
        int lDepth = maxDepth(node->left);
        int rDepth = maxDepth(node->right);

        /* use the larger one */
        if (lDepth > rDepth)
            return(lDepth + 1);
        else return(rDepth + 1);
    }
}
```

Mirror image



Mirror Trees

(1) Call Mirror for left-subtree i.e., Mirror(left-subtree)
(2) Call Mirror for right-subtree i.e., Mirror(right-subtree)

(3) Swap left and right subtrees. temp = left-subtree left-subtree = right-subtree right-subtree = temp

```
void mirror(struct Node* node)
{
  if (node==NULL)
    return;
  else
  {
    struct Node* temp;

    /* do the subtrees */
    mirror(node->left);
    mirror(node->right);

    /* swap the pointers in this node */
    temp       = node->left;
    node->left  = node->right;
    node->right = temp;
  }
}
```

Construct all possible BSTs for keys 1 to N

```
For example, for N = 2, there are 2 unique BSTs
     1               2
      \             /
       2           1

For N = 3, there are 5 possible BSTs
  1              3        3         2      1
    \           /        /        /  \      \
     3        2         1        1    3      2
    /       /            \                    \
   2      1               2                     3
```

- All node in left subtree are smaller than root and in right subtree are larger than root.
- So if we have ith number as root, all numbers from 1 to i-1 will be in left subtree and i+1 to N will be in right subtree.
- If 1 to i-1 can form x different trees and i+1 to N can from y different trees then we will have x*y total trees when ith number is root.
- N choices for root, so iterate from 1 to N for root and another loop for left and right subtree.